

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский технический университет связи и информатики»**

Кафедра «Программная инженерия»

Лабораторная работа №1
по дисциплине «Структуры и алгоритмы обработки данных»
по теме «Быстрый доступ к данным»

Выполнил студент группы:
БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Борзов В.М.

Москва, 2026

Оглавление

Введение	3
Выполнение	4
Задание 1. Создание проекта	4
Задание 2. Доступ по индексу и линейный поиск	4
Задание 3. Доступ по ключу	4
Задание 4. Исследование функции hash()	5
Задание 5. Простая хеш-таблица методом цепочек	5
Задание 6. Коэффициент заполнения и коллизии	6
Задание 7. Представление бинарной кучи	6
Задание 8. Реализация sift up и вставки	6
Задание 9. Реализация sift down и извлечения корня	7
Задание 10. Приоритетная очередь	8
Задание 11. Тор-К с полной сортировкой	8
Задание 12. Тор-К с помощью кучи	8
Задание 13. Самостоятельная работа	8
Вывод	9
Контрольные вопросы	10

Введение

Целью данной лабораторной работы является изучение влияния способа организации данных на скорость выполнения основных операций, освоение доступа к данным по индексу и по ключу, понимание принцип работы хеш-таблиц и причины возникновения коллизий, обучение работы с бинарной кучей и приоритетной очередью, а также реализация поиск Top-K элементов без полной сортировки данных.

В ходе выполнения лабораторной работы необходимо:

- 1) сравнить стоимость различных способов доступа к данным;
- 2) реализовать линейный поиск и простую хеш-таблицу с обработкой коллизий;
- 3) исследовать влияние коэффициента заполнения;
- 4) самостоятельно реализовать операции sift up и sift down для бинарной кучи;
- 5) сравнить два способа поиска нескольких наибольших элементов.

Выполнение

Задание 1. Создание проекта

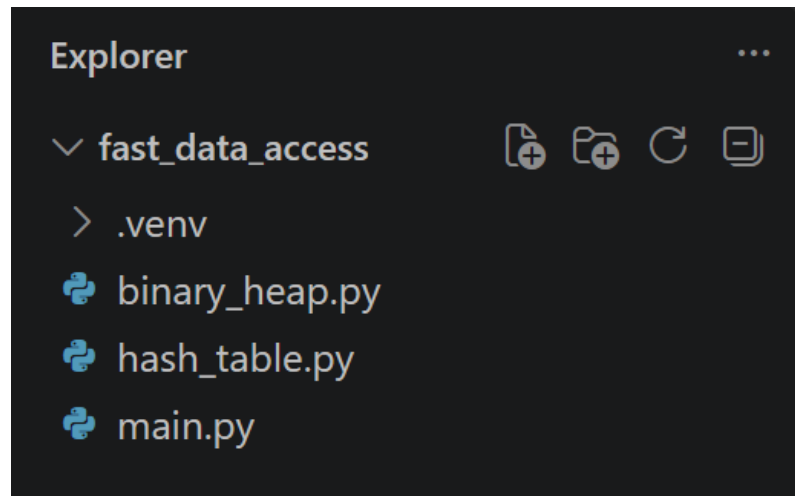


Рисунок 1 — Структура проекта

```
1 import timeit as tmt
2 # from time import perf_counter
3 from random import randint
4 import heapq as hp
5 from hash_table import HashTable
6 from binary_heap import MinHeap
7
8 BIG_NUM = int(1e5) # >=1e5
9 A = int(1e2)
```

Листинг 1 — Список импортов и константы

Задание 2. Доступ по индексу и линейный поиск

```
1 def task2():
2     def linear_search(numbers, target):
3         for i in range(len(numbers)):
4             if numbers[i] == target:
5                 return i
6         return -1
7
8     array_int = [randint(-A, A) for i in range(BIG_NUM)]
```

Листинг 2 — Задание 2

Задание 3. Доступ по ключу

```
1 def task3():
2     def id_search(values, target):
3         for val in values:
4             if val[0] == target:
```

```

5         return target
6     return -1
7
8     array_dict = [(i, randint(-A, A)) for i in range(BIG_NUM)]
9     st = 0
10    md = BIG_NUM // 2
11    fn = BIG_NUM - 1
12    id_search(array_dict, st)
13    id_search(array_dict, md)
14    id_search(array_dict, fn)
15
16    new_dict = dict(array_dict)

```

Листинг 3 — Задание 3

Задание 4. Исследование функции hash()

```

1 def task4():
2     print('hash({})={}, hash({})={}, hash({})=\nhash({})={}, \n
      hash({})={}'.format(
3         0, hash(0),
4         5, hash(5),
5         -5, hash(-5),
6         'aaaaaa', hash('aaaaaa'),
7         'uwu', hash('uwu')
8     ))
9
10    def bucket_index(key, table_size):
11        return hash(key) % table_size

```

Листинг 4 — Задание 4

Задание 5. Простая хеш-таблица методом цепочек

```

1 def task5():
2     pass

```

Листинг 5 — Задание 5

```

1 class HashTable():
2     def __init__(self):
3         self.bucket_num = 0
4         self.empty_array_list = []
5
6     def set(self, key, value):
7         pass
8
9     def get(self, key):
10        pass
11
12    def remove(key):
13        pass

```

Листинг 6 — Класс хэш-таблицы (методы set, get, remove)

Задание 6. Коэффициент заполнения и коллизии

```
1 def task6():
2     pass
```

Листинг 7 — Задание 6

```
1 class HashTable():
2     def __init__(self):
3         self.bucket_num = 0
4         self.empty_array_list = []
5         self.elem_counter = 0
6
7     def load_factor(self):
8         if self.bucket_num > 0:
9             return self.elem_counter / self.bucket_num
10        return 0
11
12    def collision_count(self):
13        pass
```

Листинг 8 — Класс хэш-таблицы (методы load_factor, collision_count)

Задание 7. Представление бинарной кучи

```
1 def task7():
2     array = [2, 5, 7, 9, 11, 10, 15]
3     heap = MinHeap(array)
```

Листинг 9 — Задание 7

```
1 class MinHeap():
2     def __init__(self, args=None):
3         self.data = list(args) if args else []
4         if self.data:
5             for i in range(len(self.data) // 2 - 1, 1, -1):
6                 self._sift_down(i)
7
8     def get_parent(self, index):
9         return (index - 1) // 2
10
11    def get_left(self, index):
12        return 2 * index + 1
13
14    def get_right(self, index):
15        return 2 * index + 2
```

Листинг 10 — Код реализации sift up и вставки

Задание 8. Реализация sift up и вставки

```
1 def task8():
2     pass
```

Листинг 11 — Задание 8

```

1 class MinHeap():
2     def __init__(self, args=None):
3         self.data = list(args) if args else []
4         if self.data:
5             for i in range(len(self.data) // 2 - 1, 1, 1):
6                 self._sift_down(i)
7
8     def _sift_up(self, index):
9         while index > 0:
10            parent = self.get_parent(index)
11            if self.data[parent] <= self.data[index]:
12                break
13            self.data[index], self.data[parent] = self.data[parent],
14            self.data[index]
15            index = parent
16            print(self.data)
17
18     def push(self, value):
19         pass

```

Листинг 12 — Код реализации sift up и вставки

Задание 9. Реализация sift down и извлечения корня

```

1 def task9():
2     pass

```

Листинг 13 — Задание 9

```

1 class MinHeap():
2     def __init__(self, args=None):
3         self.data = list(args) if args else []
4         if self.data:
5             for i in range(len(self.data) // 2 - 1, 1, 1):
6                 self._sift_down(i)
7
8     def _sift_down(self, index):
9         size = len(self.data)
10        while True:
11            left = self.get_left(index)
12            right = self.get_right(index)
13            smallest = index
14            if left < size and self.data[left] < self.data[smallest]:
15                smallest = left
16            if right < size and self.data[right] < self.data[smallest]:
17                smallest = right
18            if smallest == index:
19                break
20            self.data[index], self.data[smallest] = self.data[smallest],
21            self.data[index]
22            index = smallest
23            print(self.data)
24
25     def pop(self):
26         if not self.data:
27             print('error: empty heap')
28         else:

```

```

28         root = self.data[0]
29         last = self.data.pop()
30         if self.data:
31             self.data[0] = last
32             self._sift_down(0)
33         return root

```

Листинг 14 — Код реализации sift down и извлечения корня

Задание 10. Приоритетная очередь

```

1 def task10():
2     heap = []
3     hp.heapify(heap)

```

Листинг 15 — Задание 10

Задание 11. Тор-К с полной сортировкой

```

1 def task11():
2     array_int = [randint(-A, A) for i in range(BIG_NUM)]
3     for K in [10, 100, 1000]:
4         pass

```

Листинг 16 — Задание 11

Задание 12. Тор-К с помощью кучи

```

1 def task12():
2     def top_k_heap(data):
3         pass

```

Листинг 17 — Задание 12

Задание 13. Самостоятельная работа

```

1 def task13():
2     contestants = []

```

Листинг 18 — Задание 13

Вывод

В ходе лабораторной работы:

- было получено объяснение, почему способ организации данных влияет на скорость выполнения операций;
- были найдены различия доступа по известному индексу и последовательный поиск значения;
- был использован словарь для доступа по ключу и объяснена его связь с хеш-таблицей;
- было объяснено назначение хеш-функции и причина возникновения коллизий;
- была реализована простая хеш-таблицу методом цепочек;
- был вычислен коэффициент заполнения хеш-таблицы и проанализировано его влияние;
- была представлена бинарная куча в массиве и вычислены индексы родителя и потомков;
- были реализованы операции sift up и sift down;
- была использована бинарная куча как основа приоритетной очереди;
- было объяснено различие между полной сортировкой и поиском Top-K;
- была оценена сложность основных операций и выбрана структура данных исходя из требуемых операций.

Контрольные вопросы

- 1) *Почему выбор структуры данных влияет на скорость программы?*
Скорость операций поиска, добавления и удаления во многом зависит от того, как именно организован доступ к элементам в структуре данных.
- 2) *Что означает асимптотическая сложность $O(1)$?*
Оценка $O(1)$ означает, что количество основных шагов алгоритма не растёт пропорционально размеру набора данных.
- 3) *Почему доступ к элементу списка по известному индексу считается операцией $O(1)$?*
В списке Python элементы располагаются в определенном порядке и имеют индексы. Если индекс известен, доступ к элементу выполняется напрямую.
- 4) *Как работает линейный поиск и какова его сложность в худшем случае?*
Простейший линейный поиск последовательно сравнивает элементы с искомым значением. В худшем случае алгоритму приходится просмотреть весь список, следовательно сложность $O(n)$.
- 5) *Чем доступ по индексу отличается от доступа по ключу?*
Поиск по ключу реализован на основе хеш-таблицы в среднем выполняется значительно быстрее, чем последовательный. Но из-за дополнительной памяти и вычисления хеша ключа его внутреннее устройство сложнее.
- 6) *Для каких задач удобно использовать словарь `dict`?*
Если имеется у набора объектов имеются некие индивидуальные ключи.
- 7) *Что такое хеш-функция?*
Функция, реализующая алгоритм хэширования – способа преобразования ключа в числовое значение, которое используется для определения позиции элемента в хеш-таблице.
- 8) *Для чего хеш ключа преобразуется в индекс корзины?*
Чтобы находить, добавлять и удалять данные за постоянное время (обрабатывается цепочка элементов, а не все элементы таблицы).
- 9) *Что называется коллизией в хеш-таблице?*
Коллизией называется ситуация, когда разные ключи после применения хеш-функции попадают в одну и ту же позицию таблицы.
- 10) *Почему коллизии нельзя считать ошибкой реализации?*
Коллизии являются нормальным свойством хеш-таблиц, и полностью исключить их для произвольного множества ключей обычно невозможно.

- 11) *В чём заключается метод цепочек?*
Каждая ячейка таблицы хранит не один элемент, а список пар ключ–значение. Для поиска элемента сначала вычисляется индекс корзины, после чего просматриваются только элементы соответствующей цепочки.
- 12) *Что показывает коэффициент заполнения хеш-таблицы?*
Отношение числа сохраненных элементов к количеству корзин.
- 13) *Как увеличение коэффициента заполнения может повлиять на поиск?*
По мере увеличения коэффициента заполнения вероятность длинных цепочек возрастает, и поиск начинает требовать больше сравнений.
- 14) *Что такое бинарная куча?*
Структура данных, представляющая собой почти полное бинарное дерево и обычно хранящаяся в обычном массиве последовательно по уровням.
- 15) *Какое свойство выполняется в min-heap?*
Структура данных, представляющая собой почти полное бинарное дерево и обычно хранящаяся в обычном массиве
- 16) *Почему минимальный элемент min-heap всегда находится в корне?*
Так как значение каждого родителя не больше значений его потомков.
- 17) *Как вычислить индекс родителя элемента кучи?*
$$p = (i - 1) / 2.$$
- 18) *Как вычислить индексы левого и правого потомков?*
$$l = 2 * i + 1; r = 2 * i + 2.$$
- 19) *Для чего используется операция sift up?*
Для восстановления свойства кучи после добавления нового элемента (т.к. он сначала помещается в конец массива) его последовательно сравнивают с родителем и при необходимости меняют их местами.
- 20) *Для чего используется операция sift down?*
Для восстановления свойства кучи после извлечения корневого элемента (т.к. на его место переносится последний элемент массива) его замену последовательно сравнивают с потомками и при необходимости опускают вниз.
- 21) *Какова сложность вставки элемента в бинарную кучу?*
Сложность составляет $O(\log_N)$.
- 22) *Какова сложность извлечения корня бинарной кучи?*
Сложность составляет $O(\log_N)$.

- 23) *Что такое приоритетная очередь?*
Структура данных, где у каждого добавленного элемента есть свой уровень приоритета, от которого зависит порядок обработки.
- 24) *Чем приоритетная очередь отличается от обычной очереди FIFO?*
Обычная очередь обрабатывает элементы в порядке поступления. В приоритетной очереди следующим извлекается элемент с наиболее высоким приоритетом, независимо от того, когда он был добавлен.
- 25) *Как модуль `heapq` интерпретирует вершину кучи?*
В стандартном модуле `heapq` первым извлекается элемент с наименьшим значением. Если приоритет задается числом, необходимо заранее определить, означает ли меньшее число более высокий приоритет.
- 26) *Что означает задача Тор-К?*
Во многих задачах требуется найти не полный отсортированный набор, а только K лучших элементов.
- 27) *Почему для поиска Тор-К не всегда требуется сортировать весь набор данных?*
Куча гарантирует только положение корневого элемента и выполнение свойства кучи.
- 28) *Как работает алгоритм Тор-К с `min-heap` размером K ?*
Если K значительно меньше n , можно поддерживать `min-heap` размером K . После обработки всех данных в куче останутся K наибольших значений.
- 29) *Какова сложность Тор-К через кучу размера K ?*
Сложность составляет $O(n \log_K)$, т.к. для каждого нового элемента выполняется не более одной операции над кучей размера K .
- 30) *Почему элементы внутри бинарной кучи не обязаны быть полностью отсортированы?*
Бинарная куча по определению имеет определенный порядок хранения элементов.
- 31) *В каком случае полная сортировка может быть проще и оправданнее, чем поддержание кучи?*
При малом количестве элементов поддержание кучи имеет большую сложность, чем полная сортировка.
- 32) *Какие компромиссы между скоростью и памятью возникают при использовании хеш-таблиц?*
Быстрый доступ достигается за счет дополнительной памяти и вычисления хеша ключа.